



A population-based algorithm for solving linear assignment problems with two objectives



Xavier Gandibleux^{a,*}, Hiroyuki Morita^b, Naoki Katoh^c

^a Faculty of Sciences and Technologies/IRCCyN UMR CNRS 6597 – Université de Nantes, 2 Rue de la Houssinière BP 92208, F44322 Nantes Cedex 03, France

^b College of Sustainable System Sciences – Osaka Prefecture University, Sakai, Osaka 599-8231, Japan

^c Graduate School of Engineering – Kyoto University, Kyoto 606-8501, Japan

ARTICLE INFO

Article history:

Received 16 October 2015

Received in revised form

18 June 2016

Accepted 7 July 2016

Available online 9 July 2016

Keywords:

Multiobjective optimization

Linear assignment problem

Metaheuristic

Heterogeneous functions

ABSTRACT

The paper presents a population-based algorithm for computing approximations of the efficient solution set for the linear assignment problem with two objectives. This is a multiobjective metaheuristic based on the intensive use of three operators – a local search, a crossover and a path-relinking – performed on a population composed only of elite solutions. The initial population is a set of feasible solutions, where each solution is one optimal assignment for an appropriate weighted sum of two objectives. Genetic information is derived from the elite solutions, providing a useful genetic heritage to be exploited by crossover operators. An upper bound set, defined in the objective space, provides one acceptable limit for performing a local search. Results reported using referenced data sets have shown that the heuristic is able to quickly find a very good approximation of the efficient frontier, even in situation of heterogeneity of objective functions. In addition, this heuristic has two main advantages. It is based on simple easy-to-implement principles, and it does not need a parameter tuning phase.

© 2017 Published by Elsevier Ltd.

1. Introduction

MultiObjective Metaheuristics (MOMH) [19] are methods aiming to approximate with quality the set of efficient solutions for a reasonable computing time. These methods are also useful for exact methods, e.g. in delivering a high quality lower bound set [11], which is a helpful information for example to prune with a guarantee the unfruitful solutions. The MOMH presented in this paper is a population-based algorithm for MultiObjective Combinatorial Optimization (MOCO) problems. Here, the linear assignment problem with two objectives is considered.

The first components of this algorithm have been designed and discussed some years ago [15–18]. However and almost at the same time, solving algorithms aiming to compute an exact complete set of non-dominated points for bi-objective linear assignment problems have been introduced by ourself and others authors [8,28,29]. With the success of exact algorithms since mid-2000s, the developments of approximation algorithms based on MOMH have therefore diminished relevance for this class of problems. The finalization of the proposed algorithm in this paper has been postponed until now.

The recent research stream about a notion of “complexity”, to understand as a difficulty factor brought by the heterogeneity of objective functions [14], gives new meaning to this MOMH. Indeed, some datasets may cause difficulties during the solving when they are processed by an exact algorithm (e.g. epsilon constraint method, two phase method, branch and bound method). We observed difficulties encountered by an exact algorithms [29] on the heterogeneous instances, while in the meantime, we observed the very good behavior of our population based algorithm. Thus the proposed MOMH remains relevant for approximating the non-dominated points in such a situation. Then we revisited the design and the implementation of our population based algorithm, redone a full numerical experiment on (1) homogeneous instances and (2) heterogeneous instances presenting pathologies, and reported the investigation in this paper.

1.1. The linear assignment problem with two objectives

The linear assignment problem (LAP) is a well-known fundamental combinatorial optimization problem, where the goal is to find an optimal assignment of resources to tasks, without assigning a resource more than once, and ensuring that all tasks are completed. Several practical situations can be formulated as an LAP. In addition, LAP is a sub-problem of more complicated ones, including transportation problems, distribution problems and traveling salesman problems. Efficient specific algorithms exist to

* Corresponding author.

E-mail addresses: Xavier.Gandibleux@univ-nantes.fr (X. Gandibleux), morita@eco.osakafu-u.ac.jp (H. Morita), naoki@archi.kyoto-u.ac.jp (N. Katoh).
URL: <http://www.univ-nantes.fr/gandibleux-x> (X. Gandibleux).

solve *LAP* involving a single objective, such as the Hungarian method [22] or the successive shortest path method [1].

In a multiple objective framework, the assignment problem with two objectives, denoted *biLAP*, can be formulated as follows, where c_{il}^k are non-negative integers and $x = (x_1, \dots, x_m)$:

$$\left[\begin{array}{ll} \text{"min"} z_k(x) = \sum_{i=1}^n \sum_{l=1}^n c_{il}^k x_{il} & k = 1, 2 \\ \sum_{i=1}^n x_{il} = 1 & l = 1, \dots, n \\ \sum_{l=1}^n x_{il} = 1 & i = 1, \dots, n \\ x_{il} \in \{0, 1\} \end{array} \right] \quad \text{biLAP}$$

An objective function may reflect a notion of costs, times, distances, or again a quality of completion of tasks. Let $x, x^* \in X$ be two feasible solutions of a *biLAP*, where X denotes the *decision space*, and $z_1(x), z_2(x)$ denote two objective functions defined on X . Let $z(x) = (z_1(x), z_2(x))$ be the objective vector associated with the decision variable x , and let $Y = \{z(x) | x \in X\}$ be the *objective space*. An objective vector $z(x)$ is called a *point* of Y . A solution x^* is *efficient* if no other feasible solution x exists, such that $z_k(x) \leq z_k(x^*)$, $k = 1, 2$ with at least one strict inequality. $z(x^*)$ is a *non-dominated point* in the objective space Y . The set of efficient solutions is denoted by X_E , and the representation of X_E in Y is called the *efficient frontier*.

The set of *supported efficient solutions*, denoted by X_{SE} , is a subset of X_E , such that $z(x)$ for $x \in X_{SE}$ is on the lower-left boundary of the convex hull for $\{z(x) : x \in X\}$. For $\lambda = (\lambda_1, \lambda_2)$ with $\lambda_1 + \lambda_2 = 1$ and $\lambda_k > 0$, let $biLAP_\lambda$ denote the parametrized single objective problem: $\min \{\lambda_1 z_1(x) + \lambda_2 z_2(x) : x \in X\}$. The set X_{SE} is composed of $x \in X$ such that x is optimal to $biLAP_\lambda$ for some $\lambda = (\lambda_1, \lambda_2)$. The computation of each $biLAP_\lambda$ for a fixed λ can be implemented easily by relaxing the integrality constraints (because the property of total unimodularity of the constraint matrix) and solving the problem using a parametric simplex method, or any specific solving algorithm. Consequently, X_E is partitioned into two subsets, the set X_{SE} and the set $X_{NE} = X_E \setminus X_{SE}$ of *non-supported efficient solutions*. The sets X_E, X_{SE} , and X_{NE} are respectively denoted in the objective space by Y_N (non-dominated points), Y_{NS} (supported non-dominated points), and Y_{NN} (non-supported non-dominated points). See [10] for more details.

1.2. Classification of efficient solutions

Efficient solutions can be characterized and classified into several different subsets.

- The *complete set* (X_E) and the *minimal complete set* (X_{Em}) of efficient solutions. Generally, several distinct efficient solutions x_1^*, x_2^*, x_3^* can correspond to the same non-dominated point $z(x_1^*) = z(x_2^*) = z(x_3^*)$ in the objective space. The solutions x_1^*, x_2^*, x_3^* are said to be *equivalent* in the objective space. The number of such equivalent solutions is important; the enumeration of all of them may be intractable. This is the case for some instances of the biobjective shortest path problem in which all feasible solutions are efficient [21]. In this situation, it is impossible to design an efficient algorithm for computing all feasible solutions of X which correspond to a non-dominated point in the objective space Y .

Hansen [21] introduced the *minimal complete set* X_{Em} of efficient solutions (i.e. the subset of X_E that contains no equivalent solutions, and for any $x \in X_E$ there exists $x' \in X_{Em}$ such that x and x' are equivalent).

- The *supported* (X_{SE}) and the *non-supported* (X_{NE}) solutions. If a convex hull is computed in the objective space for all solutions in X_E , then the supported solutions belong to the boundary of

this convex hull. Any supported solution can be computed by solving the $biLAP_\lambda$ for some λ . The non-supported efficient solutions are the efficient solutions not located on the boundary of the convex hull. In the biobjective case, X_{NE} solutions are located in the triangles drawn on two successive supported efficient solutions in the objective space. There is no theoretical characterization which leads to the efficient computation of X_{NE} solutions. Applying the minimal complete set definition to X_{SE} and X_{NE} gives rise to X_{SEm} and X_{NEm} , respectively.

- The supported solutions located on a vertex (X_{SE1}) and those not located on a vertex (X_{SE2}) of the convex hull. Both solution sets can be obtained by solving the $biLAP_\lambda$. Nevertheless, computing the X_{SE2} set is generally more difficult than computing X_{SE1} , because the former requires the enumeration of all optimal solutions, which minimizes $\lambda z_1(x) + (1 - \lambda)z_2(x)$ with λ given. This enumeration is not trivial in general. Obviously, applying the minimal complete set definition to X_{SE1} and X_{SE2} gives rise to X_{SE1m} and X_{SE2m} , respectively.

The papers published in the literature are sometimes unclear concerning the ability of discussed algorithms. Some authors in the literature claim that their algorithm can enumerate “all” efficient solutions. However, it is generally difficult to compute this set as mentioned before. Thus, it is important to state clearly the class of efficient solutions which can be handled by the algorithm.

1.3. Literature

According to surveys [12,13] on MOCO, the first tentative of an exact method for solving the bi-objective linear assignment problem has been published during the 1960s. In spite of the wide importance of this optimization problem on fundamental and practical aspects, it has received very few attention during the last 50 years. An up-to-date literature overview invites us to classify the existing resolution methods into three waves:

- Wave 0: the “incomplete” methods. Proposed at an earlier stage, they only deal with solutions belonging to X_{SE} , using convex combinations of objectives, or goal programming techniques [4,6]. They are tagged incomplete here as they ignore a subset of solutions.
- Wave 1: the pioneers but also “expensive” methods. Aiming to determine a complete set X_E , they make use of single objective methods and the duality properties of the assignment problem [23,32]. However, they are time consuming as stated by Tuytens et al. [30].
- Wave 2: the moderns and also “efficient” methods. Designed for computing X_{Em} , X_{Em} or simply X_E , they make use of powerful algorithms known for the single objective version of the linear assignment problem [8,28,29]. According to the numerical validation reported in these papers, the methods are generally very efficient.

The data sets used to perform numerical experiments consider random instances (all coefficients randomly generated uniformly) and correlated instances (positive or negative correlations between coefficients). Nevertheless, exact algorithms aiming to compute X_{Em} may struggle when processing some numerical instances. However, for a problem like the *biLAP*, only objective functions differ from an instance to another. This argues that the objective functions may convey a form of “complexity”. This question, intrinsic to a numerical instance, has been approached recently in terms of study of the heterogeneity regarding the objective functions [14]. Different aspects reflecting the heterogeneity have been discussed and qualified. In particular, they pointed out two aspects, related to the scaling and the landscape.

The first aspect denotes a situation when an objective function's range of values is quite different from the range for other objective functions of the problem. The second aspect underlines the fact that objective functions may differ quite considerably in basic features, as their degree of multi-modality, presence of plateaus, separability, or smoothness. These two aspects stem only from numerical values of the objective functions coefficients. In this context, Degoutin and Gandibleux [7] have introduced such heterogeneity in the objective functions. They generated instances where the coefficients are consecutively duplicated, inserting plateaus into the objective functions.

In the absence of a rigorous theory on the issue, we have observed that the presence of these heterogeneous aspects into a numerical instance can induce various pathologies on the set of non-dominated points. They can take different forms such as a huge number of equivalent solutions, or produce efficient frontiers having atypical features such as a particular shape. Several of these heterogeneous instances presenting pathologies have been identified for the *biLAP*, and we have observed the meltdown of an exact approach on these instances (see Section 3.2.5), unlike the proposed multiobjective metaheuristics. The observed behavior of the method faced to such an instance places it as a valuable alternative to the exact algorithms, at least for several classes of *biLAP*.

The literature reports very few MOMH proposed for the *biLAP*. This fact is certainly due to the existence of exact methods and the under-estimation of theoretical and computational difficulties for tackling the *biLAP*. The Multiple Objectives Simulated Annealing (MOSA) method [31] has been during many years the unique MOMH available to the *biLAP*. The results reported in [30] for an improved version of MOSA show the limit of such a general purpose meta-heuristic; MOSA quickly encounters difficulties in producing good approximations for *biLAP*. It is able to find some exact efficient solutions only for small instance sizes (up to 30×30 variables).

For a bi-objective version of a knapsack problem and a scheduling problem, the advantage of using good genetic information in an MOMH has been underlined in our previous works [15,24]. Such “genetic information” can be deduced from the (sub)set of supported efficient solutions, or from an approximation of the efficient set given by a heuristic. We presented in 2003 [16,17] a first study, using genetic information for solving the *biLAP*. Our results, compared to those produced using MOSA, have demonstrated the relevance of this approach. However, the algorithm has shown some limitations during its progression in the detection of approximated solutions.

In this paper, we present a finalized version of a population-based method [18] for assignment problems with two objectives, aiming to approximate X_{Em} . This method is influenced by several key factors related to multiobjective metaheuristic and multi-objective combinatorial optimization. It is based on the intensive use of three operators performed on a population composed of only elite solutions. Here we use the terminology, “elite solution”, to indicate a solution in the population such that no other solution dominates that solution (such a solution is also named in the literature a *potential efficient solution*). It is denoted by X_{PE} in the decision space and Y_{PE} in the objective space. The set of these solutions is named “elite solution set”.

The rest of the paper is organized as follow. The outline of the heuristic and the detailed algorithmic description are presented in Section 2. The numerical results are reported and compared with previously published results in Section 3. The last section gives a conclusion and some directions for future developments.

2. Outline of the heuristic and its algorithmic description

Two numerical instances are used throughout this section to

illustrate the main principles of the heuristic. The first instance involves 5×5 variables (denoted 2AP5-1A20.dat in Section 3). The assignment costs for the objectives are:

$$c^1 = \begin{bmatrix} 13 & 14 & 7 & 2 & 11 \\ 5 & 10 & 11 & 7 & 10 \\ 7 & 19 & 9 & 16 & 19 \\ 3 & 19 & 10 & 0 & 6 \\ 12 & 9 & 2 & 4 & 15 \end{bmatrix}, \quad c^2 = \begin{bmatrix} 1 & 13 & 15 & 18 & 3 \\ 0 & 3 & 17 & 8 & 6 \\ 9 & 5 & 8 & 0 & 4 \\ 18 & 19 & 3 & 19 & 7 \\ 2 & 3 & 19 & 12 & 15 \end{bmatrix}$$

This small instance is used as a didactic example. The second instance involves 100×100 variables (denoted 2AP100-1A20.dat in Section 3). This large instance is used to exhibit the behavior of the analyzed component in the objective space.

2.1. Motivations and outline of the heuristic

The heuristic exploits two characteristic elements of the *biLAP*. First, the computation of an optimal solution for the linear assignment problem is polynomial and there exist efficient polynomial-time algorithms (see e.g. [3]). Consequently, obtaining some supported solutions for the *biLAP* is easy and deriving a tight lower and upper bound set is straightforward when X_{SElm} is known (see [11]). Second, a feasible assignment may be represented by a permutation π as

$$\begin{pmatrix} 1 & 2 & \dots & n \\ \pi(1) & \pi(2) & \dots & \pi(n) \end{pmatrix}$$

which means that 1 is mapped to $\pi(1)$, 2 to $\pi(2)$, ..., n to $\pi(n)$. Consequently, encoding a solution for the *biLAP* is immediate. Section 2.2 illustrates the encoding of 2AP5-1A20.dat.

The outline of the heuristic is now described. The population is first initialized with a subset of supported solutions (routine `detectPEinit`). The three operators are a local search (routine `localSearchOverNewElites`), a crossover (routine `crossoverWithElites`) and a path-relinking (routine `pathRelinkingWithElites`). An upper bound set defined in the objective space (routine `buildUpperBoundSet`) provides an acceptable limit for performing a local search. A genetic map is derived from the elite solutions (routine `elaborateGeneticInformation`). This map is composed of roulette wheels and provides useful information for fixing certain bits by crossover operators. This genetic information is refreshed periodically. Each new elite solution is noted, so that the heuristic stops when no new elite solutions have been produced after a series of iterations (routine `noteNewSolutions`). The heuristic can also be stopped after a predefined effort (parameter `iterationMax`), possibly combined with the detection of unfruitful iterations (routine `isTheEnd?`).

The main part of the heuristic is described by Algorithm 1 (the symbols $\downarrow$, $\uparrow$ and $\updownarrow$ specify the transmission mode of a parameter to a procedure; they correspond respectively to the mode IN, OUT and IN OUT. The symbol `--|` marks the beginning of a comment line). Each iteration of the algorithm performs one crossover operation and one path-relinking operation, which generates respectively one solution, and a list of solutions. For each of these generated solutions, a local search is then performed if and only if the solution is located in the “admissible area”. All elite solutions in the neighborhood are added to the current set X_{PE} . The iteration is finished in performing again a local search operation for each elite solution newly included in X_{PE} set.

Algorithm 1. The main procedure.

Require: data : c_{ij}^k ; $i = 1, n$; $j = 1, n$; $k = 1, 2$; `iterationMax`
Ensure: X_{PE}
 --| Compute the initial elite population set $X_{PE_{init}}$

```

detectPEinit(data ↓, pelnit ↑) ; pe ← pelnit
--| Compute the nadir points
buildUpperBoundSet(pe ↓, nadirs ↑)
--| Apply a first local search on the  $X_{SE1m}$  solution set
localSearchOverNewElites(pe ↑)
--| Identify the genetic heritage and elaborate the genetic map
elaborateGeneticInformation(pe ↓, map ↑)
--| Initialize the running indicators
iteration ← 1 ; elapsedTime ← 0 ; changes ← 0 ; noMore ← 0
repeat
--| Elaborate a solution by crossover
crossoverWithElites(pe ↑, map ↓, nadirs ↓)
--| Elaborate a series of solutions by path-relinking
pathRelinkingWithElites(pe ↑, nadirs ↓)
--| Apply a local search to the new elite solutions in  $X_{PE}$ 
localSearchOverNewElites(pe ↑)
--| Refresh the genetic heritage by integrating genetic
information
--| coming from the new  $X_{PE}$  in the current map
if (iteration MOD refreshMapFrequency=0)
    elaborateGeneticInformation (pe ↓, map ↑)
end if
--| Identify the producer of new elite solutions and note
--| a series of iterations without production of new  $X_{PE}$ 
noteNewSolutions(pe ↓, changes ↑)
if (iteration MOD seekChangesFrequency=0) then
    noMore ← (changes=0 ? noMore + 1 : 0) ; changes ← 0
end if
--| Check the stopping condition
until isTheEnd?(iteration++ ↓, elapsedTime ↓, noMore ↓)

```

Due to the simplicity of the method, performing one iteration consumes much less CPU time than other conventional population-based heuristics, such as genetic algorithms, especially when the individual is not promising (in this case, no local search is performed). Consequently, the heuristic can be very aggressive in implementing a generation process that performs many iterations. The approximation set contains only elite solutions. The algorithm maintains X_{PE} , and iteratively improves X_{PE} towards X_E (the set of exact efficient solutions). Thus, a poor solution, one that is far from the exact efficient frontier and/or not efficient for the current iteration, has few chance to be introduced in the approximation. In any time, the heuristic reports only a good approximation of the efficient frontier.

In contrast with other multiobjective evolutionary algorithms [5], our heuristic performs no direction searches to drive the approximation process, and it does not require any ranking method (there is no fitness measure). This is a remarkable fact, given that direction searches and ranking are often criticized; the former for guiding the heuristic search, and the latter for provoking increased computing effort. The following subsections describe the main aspects of the heuristic in more detail.

2.2. The initial population

X_{SE1m} , the minimal complete set of solutions located on the vertices of the convex hull, is computed by solving a series of $biLAP_\lambda$, the parametric problem. The resolution principle follows a dichotomic scheme (Algorithms 2 and 3). The single objective assignment problem is solved using the *Successive Shortest Path* algorithm (routine `succShortestPath`). Details concerning this algorithm are available in [1].

It is important to notice that several efficient solutions can be

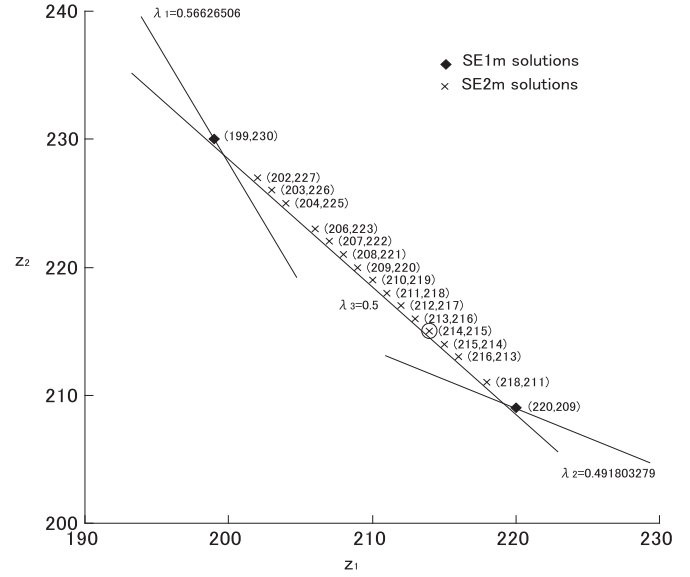


Fig. 1. Example (instance 2AP100-1A20.dat) where the optimal solution $\bar{x}$ for $biLAP_\lambda$ with $\lambda = 0.5$ points out $z(\bar{x}) = (214, 215)$, a solution in X_{SE2m} .

obtained from X_{SE2m} set using this resolution principle, as illustrated in Fig. 1 (the encircled solution). Obviously, these solutions are integrated into the initial elite population set, denoted by X_{PEinit} . However, no specific procedure has been developed for computing solutions belonging to X_{SE2m} .

An assignment is coded as a permutation of tasks 1, 2, ..., n at positions 1, 2, ..., n . For example, the coded solution $x = (4, 2, 1, 5, 3)$ means task 4 is assigned to position 1 ($x_{1,4} = 1$), task 2 to position 2 ($x_{2,2} = 1$), etc. The initial population for the didactic problem is reported in Table 1. These three solutions belong only to X_{SE1m} . The X_{SEm} is computed with the algorithm proposed by Przybylski et al. [29] for all instances used in Section 3. This information has been used for evaluating the heuristic's detection ratio.

The section concerning numerical experiments reports an analysis where the heuristic uses first X_{SEm} , and second X_{PEinit} , in order to evaluate the impact of using a poorer initial elite set.

Algorithm 2. Procedure `detectPEinit`.

```

--| Compute  $x^{(1)}$  and  $x^{(2)}$ , the optimal solutions for the two
objectives
 $x1 \leftarrow succShortestPath(c^1 \downarrow, data \downarrow)$ ;  $x2 \leftarrow succShortestPath(c^2 \downarrow, data \downarrow)$ 
 $L \leftarrow \{x1, x2\}$ 
--| Compute all intermediate solutions between  $x^{(1)}$  and  $x^{(2)}$ 
 $solveRecursion(x1 \downarrow, x2 \downarrow, data \downarrow, L \downarrow)$ 
--| Remark :  $x^{(1)}$  and  $x^{(2)}$  can be "weakly" non-dominated.
 $pelnit \leftarrow removeDsolutions(L)$ 

```

Algorithm 3. Procedure `solveRecursion`.

```

--| Compute the optimal solutions  $x^{(C)}$  of  $(P_\lambda)$  :
--|  $\min \{ \lambda_1 z_1(x) + \lambda_2 z_2(x) | x \in X \}$ 

```

Table 1

The didactic example has three solutions in the X_{SE1m} set.

X_{SE}	1	2	3	4	5	z_1	z_2
1	4	2	1	5	3	27	56
2	5	1	4	3	2	51	9
3	4	1	3	5	2	31	36

```

-| where  $\lambda_1 = z_2(x^{(A)}) - z_2(x^{(B)})$ , and  $\lambda_2 = z_1(x^{(B)}) - z_1(x^{(A)})$ .
 $c_{ij}^{\lambda} = \lambda_1 c_{ij}^1 + \lambda_2 c_{ij}^2$ ,  $i=1, n$   $j=1, n$ 
 $xC \leftarrow \text{succShortestPath}(c^{\lambda} \downarrow, \text{data} \downarrow)$ 
if ( $z(xC) \neq z(xA)$ ) and ( $z(xC) \neq z(xB)$ ) then
   $L \leftarrow L \cup \{xC\}$ 
   $\text{solveRecursion}(xA \downarrow, xC \downarrow, \text{data} \downarrow, L \uparrow)$ ;
   $\text{solveRecursion}(xC \downarrow, xB \downarrow, \text{data} \downarrow, L \uparrow)$ 
end if

```

2.3. The upper bound set

The upper bound set is defined as the set of “local nadir points”, where one nadir point is derived from two adjacent supported solutions. More precisely, if x_1 and x_2 are two adjacent supported solutions in the objective space, the corresponding nadir is a point in Y with the following coordinates:

$$(\max(z_1(x_1), z_1(x_2)), \max(z_2(x_1), z_2(x_2)))$$

Fig. 2 illustrates the upper bound set for the didactic example.

The upper bound set is used in a heuristic strategy to decide whether or not a solution is a candidate for an intensive search in its neighborhood (see also [27] for the definition of heuristic variants of this bound set). This strategy helps to save computing effort; when a solution is not dominated by the nadirs belonging to the bound set, it is located in the admissible area. Then this solution is checked to see if it is a new elite solution. All solutions in this area are considered to be promising for finding new elite solutions. A local search is then performed starting from such a promising solution. Otherwise no effort is spent on solutions outside of the admissible area.

2.4. The local search operator

We adopted a conventional neighborhood structure. Starting from a solution x , we consider the swap move (i_{j1}, i_{j2}) between two tasks. The set of pairwise exchanges (i_{j1}, i_{j2}) , with $j1 = 1, \dots, n-1$

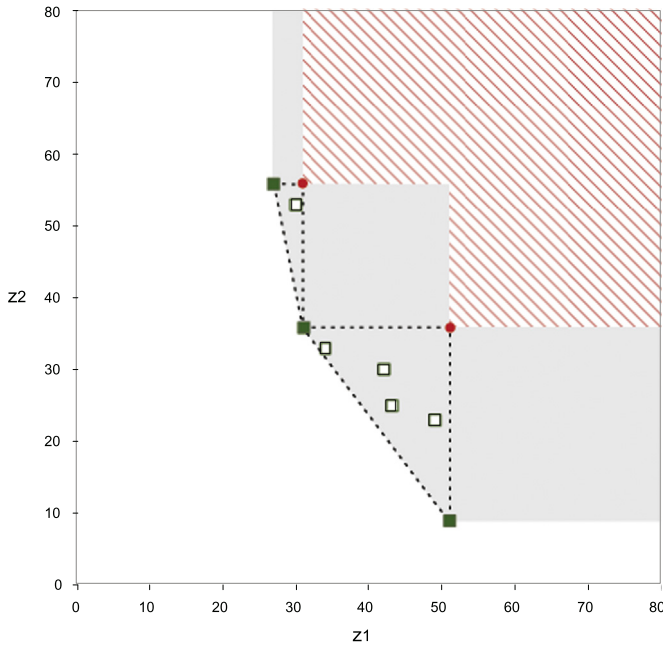


Fig. 2. Efficient solutions (squares), upper bound set (bullets) and the admissible area (grey) where a local search procedure will be performed on new solutions. Triangles with dashed lines denote areas where efficient solutions can exist. 2AP5–1A20.dat.

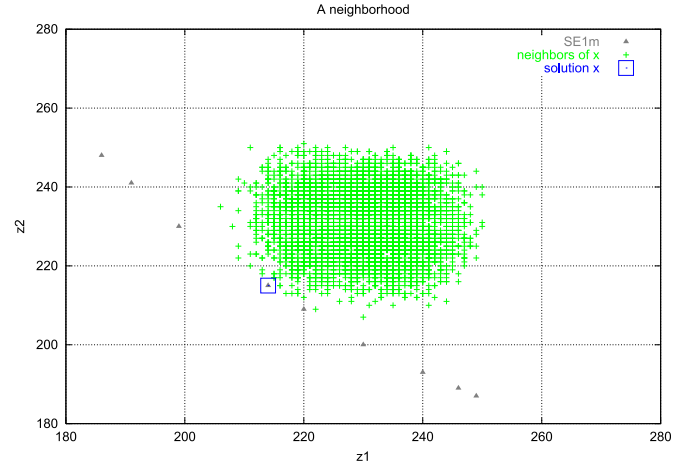


Fig. 3. The neighbors of one solution $x \in X_{SE1m}$ for the 2AP100–1A20.dat instance.

and $j2 = j1 + 1, \dots, n$, defines an associated neighborhood $\mathcal{N}(x)$ for the current solution x . The local search is described in Algorithm 4. An example of neighbors generated is given in Fig. 3. Because the computational cost for a local search can be significant ($O(n^2)$), the local search is triggered only for promising candidate solutions resulting from the crossover and the path-relinking operators. The local search routine is also called in `localSearchOverNewElites` operator, which ensures that a local search is performed only once for each solution in the elite set (Algorithm 5).

Algorithm 4. Procedure `SeekInNeighborhood`.

```

for  $j1$  in  $1..n-1$  do
  for  $j2$  in  $j1 + 1..n$  do
     $I' \leftarrow \text{swap}(I_{j1}, I_{j2})$  -| giving a new neighbor
    if  $\text{isNonDominated}(I', pe)$  and ( $z(I') \neq z(I), \forall I \in pe$ ) then
      -|  $I'$  is a new unique elite solution
       $pe \leftarrow \text{Add/update } pe \text{ with } I'$ 
    end if
  end for
end for

```

Algorithm 5. Procedure `localSearchOverNewElites`.

```

-| Perform a local search on all the last added solutions in  $pe$ 
loop
   $\text{lastAdded} \leftarrow \text{set of all the last added solutions in } pe$ 
  exit when  $\text{lastAdded} = \emptyset$ 
  for all  $I \in \text{lastAdded}$  do
     $\text{seekInNeighborhood}(I, pe)$ 
  end for
end loop

```

2.5. The genetic map

The mechanism used here is inspired by the principle of pheromones in an artificial ant colony [9] or again from the use of the frequency based memory in tabu search [20]. In Evolutionary Computation literature, this mechanism is referred to as a univariate marginal distribution (over the decision variables) and is used in a different manner in several Estimation of Distribution Algorithms (see [2,25,26]).

Assuming that similarities exist among assignment vectors of efficient solutions, the occurrence frequency of assignment “ i in position j ” in vector x for elite solutions is computed and

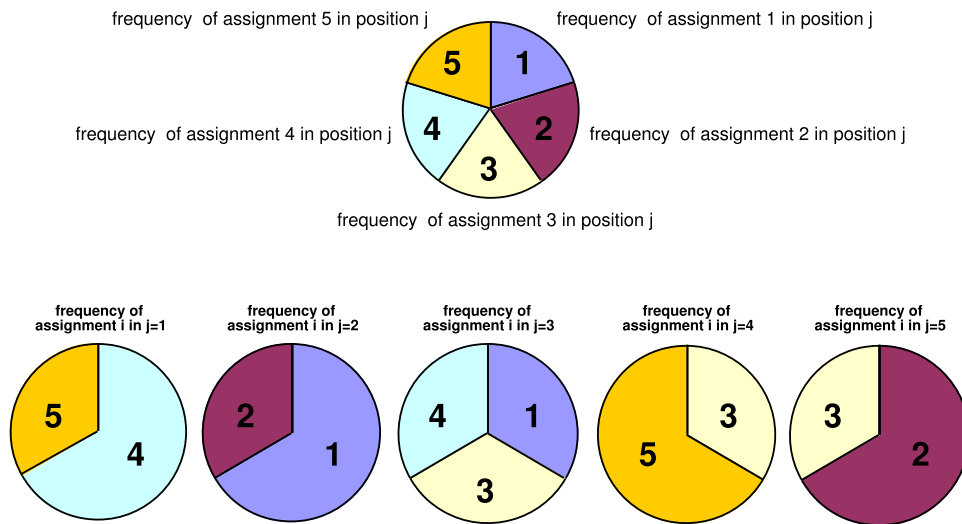


Fig. 4. First row: general example for an occurrence frequency of assignment i in position j for the set X_{PE} . Second row: the genetic map corresponding to the didactic example.

summarized in a roulette wheel that provides the genetic information. A roulette wheel is elaborated for each position, and the genetic map is made up of roulette wheels. The map is the genetic heritage of our population, and this information is used intensively by the crossover operator (Section 2.6).

The genetic information is always derived from elite solutions. The initial map is then derived only from exact supported solutions. For the didactic example, the elite set contains 3 solutions (Table 1). In the assignments observed in position 1 (second column in the table), the task 4 appears twice and the task 5 appears once. Given a uniform roulette wheel principle, the probability of selection the task i in position $j = 1$ is then $(0, 0, 0, 2/3, 1/3)$. This information is summarized on the first wheel of the genetic map which is shown in the second row of Fig. 4.

The genetic map is refreshed periodically. Roulette wheels are rebuilt using X_{PE} , the current set of elite solutions, when X_{PE} has been significantly renewed. In the current version, refreshment occurs during generation, after a predefined number of iterations (parameter `refreshMap Frequency`) has been performed. The parameter value is experimentally set at 100 000 iterations.

2.6. The crossover operator

For each crossover operation, two parent individuals are randomly selected from the current elite population, and one offspring is produced. Genes common to the parents are copied in the child. The other genes are determined using the genetic map, on the basis of the occurrence frequencies stored in the roulette wheel. To illustrate this second step, consider $I1$ and $I2$, two individuals in the didactic example, with $I3$ being the solution resulting from the crossover:

j	1	2	3	4	5
$I1$	(4 2 1 5 3)				
$I2$	(4 1 3 5 2)				
$I3$	(4 * * 5 *)				

The function `useGeneticInformation(j, map)` is called using $j = 2, 3, 5$ and the genetic map given in Fig. 4. The three genes are

then fixed as follows:

1. To determine a task i in position $j = 2$, the map's 2nd wheel suggests to choose one from the set $\{1, 2\}$ with a selection probability of $2/3$ and $1/3$ respectively. Using this information, the task is randomly selected according to the roulette principle. In this way, task 2, for example, is selected (function `selectAssignmentUsingRouletteWheel`).
2. For the position $j = 3$, the 3rd wheel suggests tasks $\{1, 3, 4\}$. However, assigning task 4 to position 3 produces an infeasible solution. Consequently, the wheel is reduced (i.e. the probabilities of non-feasible tasks are disabled while the probabilities of feasible tasks are renormalized into $[0,1]$ giving $1/2$ for tasks $\{1, 3\}$), so that it contains only feasible assignments for the position in question, in this case, tasks $\{1, 3\}$. Task 3, for example, is selected according to the roulette principle (function `selectAssignmentUsingRouletteWheel`).
3. In position $j = 5$, the wheel suggests tasks $\{2, 3\}$. But none of them is a feasible choice in this position. Therefore, the wheel is emptied, since no feasible choice exists. A list of feasible tasks (LFT) is then elaborated for this position, and one task is randomly selected. In this case, LFT contains only task $\{1\}$. Thus, the random selection of a task from the LFT results in the assignment of task 1 to position 5 (function `selectFeasibleAssignmentRandomly`).

Finally, the bound set is used to check that the solution produced $I3$ falls in the admissible area. If so, the solution is compared with the current list of elite solutions, and a local search is performed. Otherwise, the improvement strategy is not triggered, and the solution is simply ignored. Fig. 5 shows a sample of the solutions produced by this operator during the course of the algorithm for the `2AP100-1A20.dat` instance. The main steps of this operator are summarized in Algorithms 6 and 7.

Algorithm 6. Procedure `crossoverWithElites`.

```
--| Selection of individuals
Select  $I1 \in pe$  and  $I2 \in pe$  at random,  $I1 \neq I2$ 
--| An offspring  $I3$  is built from individuals  $I1$  and  $I2$  by using
--| the genetic information coming from  $I1$ ,  $I2$  and the  $X_{PE}$  set
for  $j$  in  $1..n$  do
    if  $I1_j = I2_j$  then
```

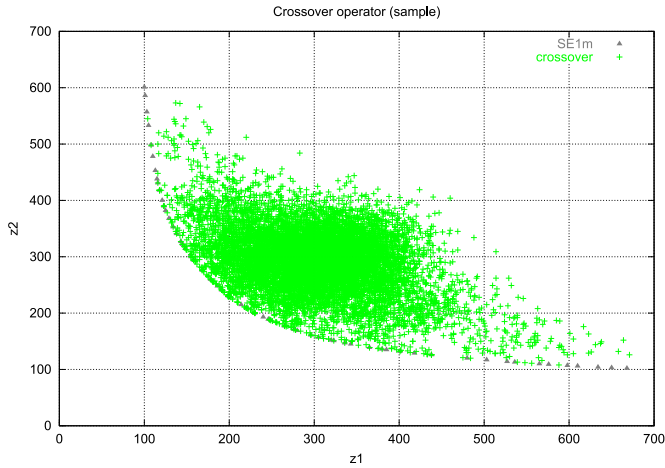


Fig. 5. Sample of the solutions produced by the crossover operator for 2AP100–1A20.dat.

```

--| Common genes are copied
I3j ← I1j
else
--| Others genes : value is selected using the wheel
I3j ← useGeneticInformation(j ↓, map ↓)
end if
end for
--| Check if I3 is a new promising sol.; if so, to perform a local
search
--| if I3 is outside of the promising area, then it is ignored
if isNonDominated(I3, nadirs) then
--| I3 is located in the admissible area for performing a LS
if isNonDominated(I3, pe) and (z(I3) ≠ z(I), ∀ I ∈ pe) then
--| I3 is a new unique elite solution
pe ← Add/update pe with I3
end if
SeekInNeighborhood(I3, pe)
end if

```

Algorithm 7. Function useGeneticInformation.

```

--| Assignment i for position j is selected using the map
composed of the roulette wheels
i ← selectAssignmentUsingRouletteWheel(j, map)
if the assignment (i, j) is not feasible then
LFT ← elaborate a list of feasible tasks for position j
i ← selectFeasibleAssignmentRandomly(LFT)
end if
return i

```

2.7. The path-relinking operator

Path-relinking generates new solutions by exploring the trajectories that connect elite solutions, starting from one solution – the initiating solution – and generating a path through the neighborhood space that leads toward the other solution – the guiding solution – [20]. Because the population contains elite solutions, the presence of a path-relinking operator in our heuristic is a natural development.

A path-relinking operation starts by randomly selecting I_1 and I_2 , two individuals in the current elite population. Because both are elites, both could potentially be the guiding solution. Let I_1 be the initiating solution and I_2 the guiding solution. The path-relinking operation generates a path $I(= I_0), I_1, \dots, I_2$, such that the

distance between I_i and I_2 decreases in i , where distance is defined as the number of different positions for tasks assigned in I_i and I_2 .

A path is generated as follows: let $i = 0$. The list LDA reports the tasks assigned to different position in solutions I_i and I_2 . A new solution is built by randomly selecting one task from LDA, and assigning this task to its final position j in I_i , which is its position in the guiding solution I_2 . In order to maintain the solution feasible, the task currently in position j in I_i is moved to j , the current position of the selected task. Let us illustrate the principle using the didactic example:

j	1	2	3	4	5
I_1	= (4)	2	1	5	3
I_2	= (4)	1	3	5	2
I_0	= (4)	2	1	5	3

LDA	= (	1	3	2	)	→	3
I_1	= (	4	2	3	5	1	)

LDA	=	(	1	2	)	→	1	
I_2	=	(	4	1	3	5	2	)

As shown above, only one solution is built in the neighborhood of I_i at each iteration, which limits the effort needed, in avoiding the examination of the other candidates in the neighborhood. The fact that the task is randomly selected from LDA introduces a form of diversity in the solutions generated along the path.

Similarly as the crossover operation, the bound set is used to check that the solution produced I_i falls in the admissible area. If so, the solution is compared with the current list of elite solutions, and a local search is performed. Otherwise, no improvement strategy is triggered, and the solution is simply ignored. Fig. 6 shows a sample of the solutions produced by this operator. The main steps of this operator are summarized in Algorithm 8.

Algorithm 8. Procedure pathRelinkingWithElites.

```

--| Selection of individuals

```

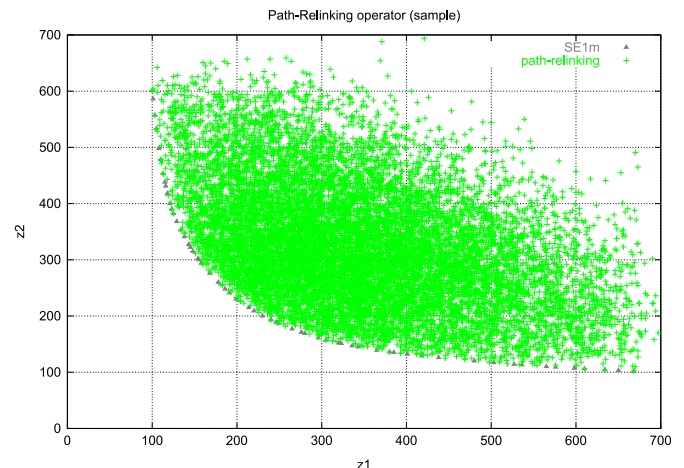


Fig. 6. Sample of the solutions produced by the path-relinking operator for the 2AP100–1A20.dat instance.

```

Select I1 ∈ pe and I2 ∈ pe at random, I1 ≠ I2
-| Elaborate a path between I1 (initiating solution) and
-| I2 (guiding solution)
while I1 ≠ I2 do
  -| Perform a move from I1 to I2
  LDA ← list of difference in assignments between I1 and I2
  i ← random(1...card(LDA))
  taskI2 ← taskOfInPosition*(I2, i)
  j ← positionOfInIndividual*(taskI2, I1)
  I1j ← taskOfInPosition*(I1, i)
  I1i ← taskI2
  -| Check if I1 updated is a new promising sol.; if so, to
  perform a
  -| local search. If I1 is not in the promising area, it is ignored
  if isNonDominated(I1, nadirs) then
    -| I1 is located in the admissible area for performing a LS
    if isNonDominated(I1, pe) and (z(I1) ≠ z(I), ∀ I ∈ pe) then
      -| I1 is a new unique elite solution
      pe ← Add/update pe with I1
    end if
    SeekInNeighborhood(I1, pe)
  end if
end while

```

2.8. The stopping rules

Three stopping rules are available in the predicate `isTheEnd?`. Each of them can be activated separately or in combination with one of the other two. The stopping rules are based on three aspects:

- **Number of iterations:** The heuristic is stopped after a predetermined effort (parameter `iterationMax`). The suggested value is 250 000 iterations.
- **A timeout:** The heuristic is stopped after a predefined elapsed time in seconds (parameter `elapsedTime`).
- **Unfruitful iterations:** After a cycle of a predetermined number of iterations (parameter `seekChangesFrequency`), the stopping rule checks to see if new elite solutions were added during the cycle (routine `noteNewSolutions`). Consecutive cycles without change are counted, and the heuristic is stopped when a predetermined number of cycles is recorded (parameter `noChangeMax`). The suggested value is 2 cycles of 100 000 iterations.

Different combinations of these stopping rules are discussed in the next section. The most realistic stopping condition is a configuration of rules combining one absolute value (number of iterations or timeout) with the detection of unfruitful iterations.

3. Numerical experiments

3.1. Experimental environment

A library of numerical instances for MOCO problems is available online.¹ This library contains a series of fifteen *biLAP* instances that we used for our experiments. The name of an instance provides the following characteristics: the number of objectives, the problem, the size, the series, the objective type, and the range of coefficients in $[1, \text{value}]$. For example, *2AP5-1A20'* is a biobjective

assignment problem with 5×5 variables from the series 1; the coefficients of the objective function are generated randomly (denoted by A) in the range $[1, 20]$.

In all of the following, the instances used are biobjective, with objective coefficients generated randomly in the range $[1, 20]$: *2AP*-1A20'*. The coefficients of an objective function are given by a square matrix $n \times n$, for all objectives and all instances. Without ambiguity and with a view to easing the presentation of tables and figures in this section, the value n which refer to the size of an instance, means that the instance is defined by $n \times n$ matrices. The instances *2AP5-1A20* through *2AP50-1A20* were also used in [30].

The computer used for the experiments is a laptop equipped² with a MacBook Pro Intel Core 2 Duo 2.53 GHz processor, and 8 GB of RAM, running under the OSX operating system, version 10.6.6. The algorithms were implemented in the C programming language. The binary code was obtained using the gcc-4.2.1 compiler with the -O3 optimizer option.

The default values of the parameters were frozen to implement a generation performing 250 000 iterations (parameter `iterationMax`), with the genetic map refreshed every 100 000 iterations (parameter `refreshMapFrequency`). Each experiment was repeated 5 times using different seed values. The min/avg/max values were recorded for each observed indicator. This experimental environment is common to all numerical experiments, unless explicitly stated otherwise.

The performance measure $M1$ is used to appreciate and compare the approximation quality of X_{PE} . This indicator was introduced by [31] to measure the ratio of exact efficient solutions contained in the elite solution set X_{PE} , i.e., $M1 = |X_{PE} \cap X_{Em}| / |X_{Em}|$.

3.2. Analyses

3.2.1. Minimal complete solution sets and initial elite solution set

The minimal complete set X_{Em} is computed on this laptop with the algorithm already mentioned [29]. This set is decomposed into the characteristic subsets, and particularly X_{SE1m} and X_{SE2m} . The initial elite population set X_{PEinit} is computed by solving an optimal assignment with the Successive Shortest Path (SSP) algorithm, for some convex combinations of the objectives. Table 2 reports all the solutions computed. The CPUt column of this table provides the time consumed (seconds) by the `detectPEinit` routine to compute X_{PEinit} on the computer used for our experiments.

According to Fig. 7, the number of solutions in each subset grows linearly with the input size for these instances. It is interesting to note that the difference between the sizes of X_{SE1m} and X_{SE2m} seems to decrease with input size.

The examination of the X_{PEinit} column confirms that X_{PEinit} is composed of all solutions belonging to X_{SE1m} , plus some additional solutions from X_{SE2m} . It is not surprising to observe an increasing difference between X_{SE1m} and the X_{PEinit} , in instances involving large input size. Because X_{SE2m} grows with input size, the frequency of detected solutions in X_{SE2m} using the dichotomic principle of the routine `detectPEinit` increases. The computation time of X_{PEinit} for all instances is small.

3.2.2. Approximations observed according to diverse stopping conditions

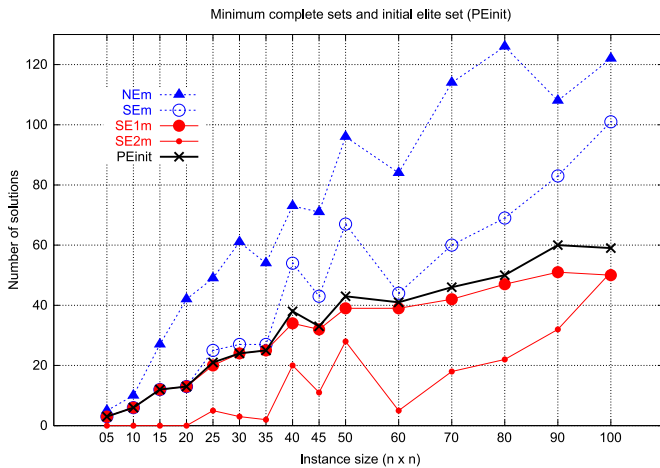
Table 3 (top, center, bottom) reports the performances of the heuristic given the respective activation of stopping rules 1, 2 and 3 (i.e. number of iterations, timeout and unfruitful iterations) as stopping conditions. We used 250 000 iterations as the

² The proposed algorithm has been tested on an old laptop (PowerPC G4 400 MHz processor; 256 MB of RAM, OS X version 10.2.6.) with an observed performance of 10 times slower on the old laptop.

¹ MOCOlib at <http://www.mcdmsociety.org/>

Table 2Distribution of efficient solutions in X_{Em} , and its subsets, as well as in X_{PEinit} .

Instance $n \times n$	[29]					SSP	
	X_{Em}	X_{NE_m}	X_{SE_m}	X_{SE1m}	X_{SE2m}	X_{PEinit}	CPUt
5	8	5	3	3	0	3	<1.0
10	16	10	6	6	0	6	<1.0
15	39	27	12	12	0	12	<1.0
20	55	42	13	13	0	13	<1.0
25	74	49	25	20	5	21	<1.0
30	88	61	27	24	3	24	<1.0
35	81	54	27	25	2	25	<1.0
40	127	73	54	34	20	38	<1.0
45	114	71	43	32	11	33	<1.0
50	163	96	67	39	28	43	<1.0
60	128	84	44	39	5	41	<1.0
70	174	114	60	42	18	46	1.1
80	195	126	69	47	22	50	1.8
90	191	108	83	51	32	60	3.0
100	223	122	101	50	51	59	4.1

**Fig. 7.** Minimal complete sets and the initial elite set (X_{PEinit}).

iterationMax parameter for stopping rule 1, 6 s as elapsed-Time parameter for stopping rule 2 (#iter reports the number of iterations performed for the time allowed), and two cycles of 100 000 iterations as the noChangeMax parameter for stopping rule 3. In each case, two situations were evaluated depending on whether the initial elite set was initialized with X_{SE_m} , the minimal complete set of supported solutions, or with the X_{PEinit} detected by the routine detectPEinit.

Initializing the set of elite solutions with a less rich population of individuals (in this case, a smaller number of solutions) does not seem to deteriorate the heuristic's ability to approximate the efficient frontier. The detection ratio curve is delayed when the population is less rich, but it preserves the same tendency. For example, when the stopping rule 1 is activated (Fig. 8), the detection ratio M1 for X_{PEinit} is generally worse compared with X_{SE_m} , and the difference increases with the input size of the instances. Nevertheless, the detection quality remains very good, with a difference of less than 10%. Consequently, if the time consumed for detecting an initial population set is judged unacceptable or if it becomes excessive with larger input sizes, the detection procedure can be stopped at a premature stage. Although the initial elite set will not be completed, the heuristic is still able to compute good approximations of efficient solutions with initially inferior genetic information.

For the instances up to 50×50 variables, the detection quality is practically identical no matter what stopping rule is used (see

Table 3

M1 (min/avg/max) measured with stopping rule 1–3 and using X_{SE_m} or X_{PEinit} (Top: M1 and CPUt (avg) for stopping rule 1; Center: M1 and #iter (avg) for stopping rule 2. #iter is the number of iterations performed by the algorithm during a generation over 5 runs; Bottom: M1 and CPUt (avg) for stopping rule 3.)

Instance $n \times n$	X_{SE_m}				X_{PEinit}			
	Min	Avg	Max	CPUt	Min	Avg	Max	CPUt
5	100.0	100.0	100.0	<1.0	100.0	100.0	100.0	<1.0
10	100.0	100.0	100.0	<1.0	100.0	100.0	100.0	<1.0
15	100.0	100.0	100.0	1.3	100.0	100.0	100.0	1.3
20	100.0	100.0	100.0	2.5	100.0	100.0	100.0	2.0
25	90.5	94.6	97.3	2.1	90.5	93.2	95.9	2.1
30	93.3	94.8	96.6	3.5	94.4	96.4	97.8	3.7
35	95.1	96.0	96.3	4.1	95.1	95.3	96.3	4.0
40	94.5	96.6	98.4	4.3	87.5	89.2	92.2	4.9
45	85.1	87.7	91.2	4.7	83.3	87.2	91.2	5.7
50	89.7	90.3	91.5	6.9	85.5	87.0	88.5	8.2
60	76.6	78.4	80.5	8.5	69.5	73.3	77.3	8.8
70	77.0	80.5	82.8	12.1	70.7	71.7	74.1	12.8
80	74.9	77.1	79.5	16.5	74.9	77.8	81.0	17.5
90	72.3	75.3	77.0	19.8	71.2	71.9	72.3	20.0
100	71.3	72.4	74.4	25.4	60.1	62.5	63.7	29.2

$n \times n$	Min	Avg	Max	#iter	Min	Avg	Max	#iter
5	100.0	100.0	100.0	5 295 096	100.0	100.0	100.0	5 080 107
10	100.0	100.0	100.0	2 951 570	100.0	100.0	100.0	2 891 947
15	100.0	100.0	100.0	1 051 004	100.0	100.0	100.0	1 022 942
20	100.0	100.0	100.0	581 343	100.0	100.0	100.0	568 462
25	95.9	97.6	98.6	709 546	95.9	96.8	98.6	676 334
30	95.5	96.0	96.6	419 831	95.5	97.1	98.9	388 096
35	96.3	96.3	96.3	368 043	95.1	95.8	96.3	359 005
40	94.5	96.7	98.4	349 506	87.5	90.2	93.8	296 375
45	87.7	88.8	91.2	314 674	83.3	87.2	91.2	252 853
50	89.7	90.2	91.5	218 105	84.2	85.3	86.7	179 193
60	71.9	74.1	75.8	180 284	66.4	71.7	75.0	167 976
70	71.8	73.9	75.3	124 785	64.9	67.5	70.1	116 007
80	67.7	71.6	73.8	92 912	70.3	71.4	72.3	85 111
90	68.1	69.2	70.7	77 062	59.7	62.8	64.9	73 591
100	61.4	64.1	66.8	59 407	48.4	49.7	50.2	50 648

$n \times n$	Min	Avg	Max	CPUt	Min	Avg	Max	CPUt
5	100.0	100.0	100.0	<1.0	100.0	100.0	100.0	<1.0
10	100.0	100.0	100.0	<1.0	100.0	100.0	100.0	<1.0
15	100.0	100.0	100.0	1.1	97.4	99.5	100.0	1.0
20	100.0	100.0	100.0	1.9	96.4	98.9	100.0	2.1
25	90.5	95.4	98.6	2.8	89.2	92.4	95.9	2.0
30	93.3	94.6	96.6	3.6	94.4	96.2	97.8	3.3
35	95.1	96.0	96.3	4.8	95.1	95.1	95.1	3.6
40	94.5	96.6	98.4	5.6	88.3	92.7	94.5	10.7
45	86.0	88.4	91.2	6.7	84.2	87.9	93.0	8.5
50	90.3	91.9	92.7	14.6	87.3	88.8	89.7	17.9
60	82.0	84.5	86.7	21.4	79.7	81.9	85.2	26.6
70	81.0	86.1	90.8	32.5	71.8	77.4	82.8	45.7
80	79.0	81.3	84.1	45.5	80.5	83.5	86.7	54.1
90	77.0	80.8	84.3	66.6	77.0	79.6	83.2	77.7
100	78.0	79.6	81.6	103.0	65.5	69.3	74.0	116.6

Fig. 9). In larger instances, the differences are visible. Three observations can be made. First, the detection curves for all three stopping rules exhibit the same behavior. Second, even given a very short computing time, our heuristic is able to find more than 50% of exact solutions for the largest instance. Third, allowing the heuristic to reiterate until there are no new candidate solutions for the set X_{PE} produces a final approximation containing a larger number of exact efficient solutions. This last observation indicates that the algorithm tends to progress towards the exact solution set, although it may require a significant amount of CPUt to reach it.

3.2.3. The impact of heuristic components on the generation process

The proposed algorithmic solution (see Algorithm 1) is built on four main components: crossover, path-relinking, local search on

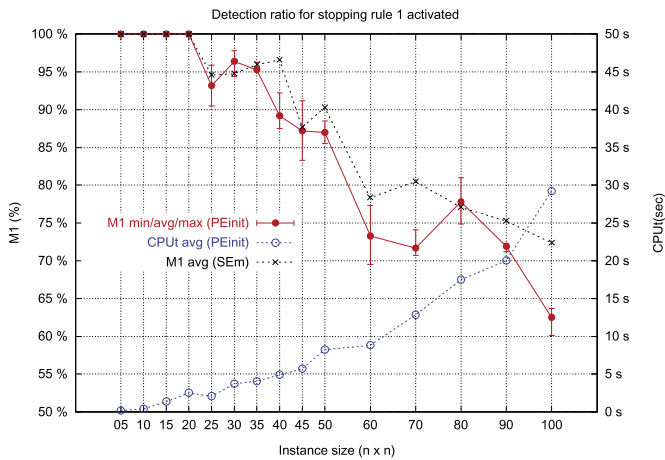


Fig. 8. M1 (min/avg/max) and CPUt (avg) on X_{SEm} or X_{PEinit} , and stopping rule 1.

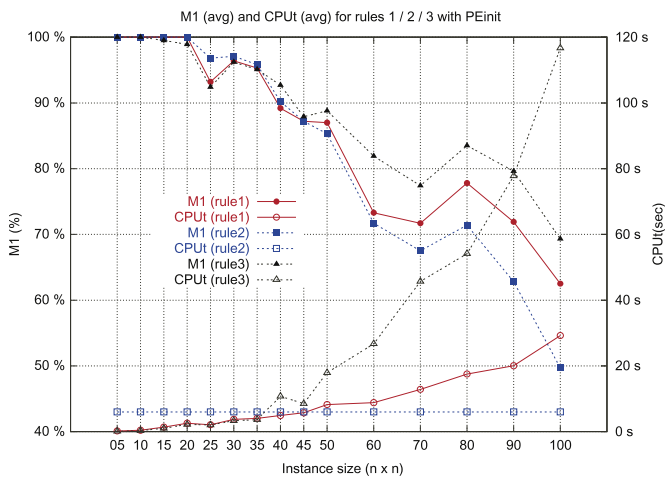


Fig. 9. M1 (avg) and CPUt (avg) on X_{SEm} and X_{PEinit} for stopping rules 1, 2 and 3.

new elites and local search in a neighborhood. The following analysis shows how each component participates in the progress of detecting exact efficient solutions. To achieve this analysis, four configurations of the heuristic were tested using 2AP100–1A20. In each configuration, one component of the heuristic was disabled. Notice that the four components are not independent, although crossover and path-relinking are independent of each other. For example, all crossover, path-relinking and local search in a neighborhood call the procedure of local search on new elites (see Algorithms 5, 6 and 8). Table 4 reports values for M1 and CPUt at the end of the heuristic (stopping rule 1 activated; 250 000 iterations). Fig. 10 shows the progression curves for the intermediate steps.

The crossover operator contributes particularly to the detection of good solutions during the early iterations of the generation process. However, crossover seems less powerful than path-relinking. Indeed, the time spent by the heuristic when crossover or

Table 4
Progression on M1 for instance 2AP100–1A20 with 250 000 iterations.

Component disabled in the heuristic	M1 (%)	CPUt (s)
None	62.33	33.60
Crossover operator	61.88	15.20
Path-relinking operator	42.15	18.60
Local search on new elites operator	61.88	33.50
Local search in a neighborhood	44.84	25.20

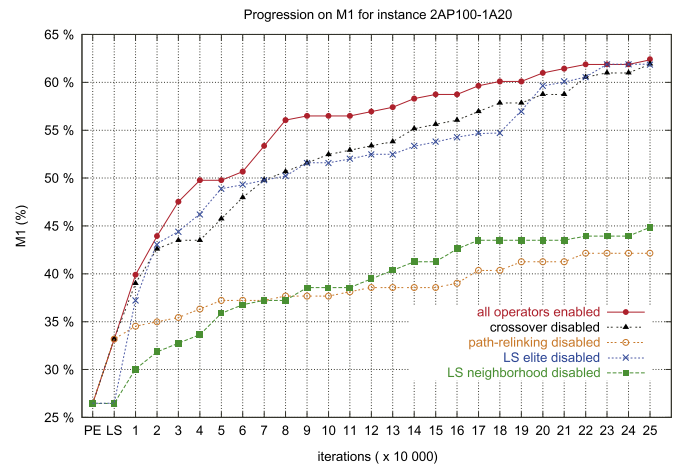


Fig. 10. Detection ratios for the cases with different sets of operators enabled.

path-relinking is disabled is of the same order (15.2 s and 18.6 s respectively). However, the heuristic is definitely less effective when path-relinking is disabled (a final M1 value of 42% compared to 62%). Analysis shows that the crossover operation contributes to the detection of new elite solutions, but its impact on detection fades as iterations increase. One explanation is that the genetic map loses its information capacity as the X_{PE} set becomes larger.

The local search on new elites pushes the good solutions towards the efficient frontier. This operator is especially effective during early iterations of the generation, because many new solutions are detected. However, it becomes much less effective in the latter half of iterations.

The pair, 'path-relinking' and 'local search in a neighborhood', is clearly a powerful instigator of solutions within the heuristic. When either of these two components is disabled, the corresponding decrease in detection power of the heuristic is similar in proportion. Together, these two components seem to be sufficient to constitute a simplified version of the heuristic. Nevertheless the 'crossover' and 'local search on new elites' components appear to be good complements for the heuristic.

3.2.4. Our final results compared with those existing in the literature

The reported computational results in column two of Table 5 are taken from the results published in [30]. They are obtained for one run only using the improved MOSA method with the same set

Table 5
MOSA vs. the population-based heuristic with stopping rules 1 and 3.

Instance $n \times n$	MOSA M1	X_{PEinit}		M1 Max	CPUt Avg	#iter Avg
		Min	Avg			
5	87.5	100.0	100.0	100.0	<1.0	100 000
10	56.2	100.0	100.0	100.0	<1.0	150 000
15	25.6	97.4	99.5	100.0	1.0	180 000
20	3.7	96.4	98.9	100.0	1.9	190 000
25	0.0	89.2	92.4	95.9	1.8	210 000
30	3.4	94.4	96.2	97.8	3.3	210 000
35	0.0	95.1	95.1	95.1	3.7	220 000
40	0.0	87.5	89.2	92.2	5.3	250 000
45	0.0	83.3	86.3	91.2	5.9	240 000
50	0.0	85.5	87.0	88.5	8.9	250 000
60	N.A.	69.5	73.3	77.3	9.8	250 000
70	N.A.	70.7	71.7	74.1	14.6	250 000
80	N.A.	74.9	77.8	81.0	20.2	250 000
90	N.A.	71.2	71.9	72.3	24.0	250 000
100	N.A.	60.1	62.5	63.7	34.7	250 000

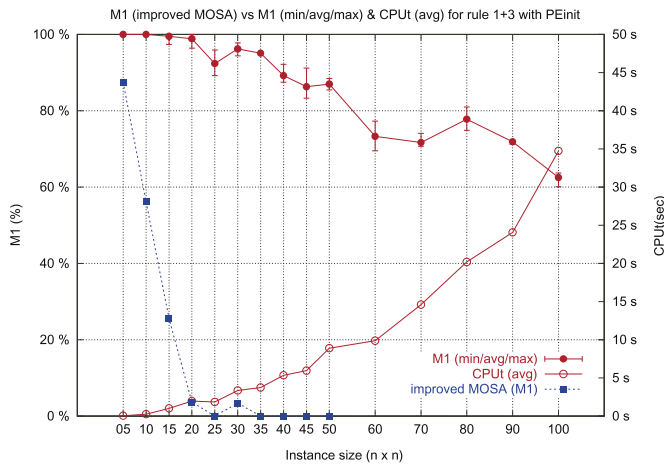


Fig. 11. The population based heuristic with stopping rules 1–3 activated, compared with MOSA.

of numerical instances. Fig. 11 shows that MOSA method performs poorly in terms of the quality measure M1, especially when the size of the instances increases. Since the computer used is not the same (MOSA results have been computed on DEC3000 alpha), a discussion in terms of CPUt is not allowed.

The columns on the right-hand side of Table 5 report the results of our heuristic when stopping rules 1 and 3 are activated. The generation is tuned to 250 000 iterations and the genetic map is refreshed each 100 000 iterations. The CPUt indicated is an average value for one complete run of the heuristic (total running time). It includes the time used for computing the initial population of elite solutions, and the time used for the approximation of the efficient frontier. The rightmost column of the table (#iter) gives the average number of iterations performed by the algorithm during a generation over 5 runs. A value other than 250 000 indicates that stopping rule 3 was triggered before stopping rule 1.

The comparison of MOSA results with those produced by the population based heuristic proposed in this paper underlines the superiority of our heuristic over MOSA (Fig. 11). We suppose that our heuristic consumes more time than the MOSA method (CPUt for MOSA are reported as 5 s and 246 s respectively for instances 5×5 and 50×50). However, our heuristic has two main characteristics that persuade us that in tests run on the same computer, our method would outperform the MOSA method.

First, the detection of solutions evolves very quickly during the early iterations of the generation. Despite the short time allowed for the generation, the approximation quality is already good (see Table 3, middle). Second, our heuristic is able to improve its approximation if more time is allowed (see Table 3, bottom). MOSA, on the other hand, does not seem to be able to improve its approximation even given more time. According to the values reported for the quality indicator M_2 (i.e. approximations located inside triangles defined by three points: two adjacent supported solutions in the objective space and the corresponding nadir) in [30], the MOSA method rapidly encounters difficulties for detecting good approximations of the efficient frontier.

3.2.5. Heterogeneous instances presenting pathologies

Six numerical instances are used for illustrating situations of difficulty encountered by an exact method for computing a minimal complete set of solutions. These data sets come from referenced papers [7,28,30]. They are described in Table 6. As reported by the numerical experiments summarized in Table 7, the two phase method which is very efficient on Tuytens instances, has difficulties for solving the Pedersen instances and is not able to

Table 6

Six selected numerical instances: characteristics.

Name	$n \times n$	Range	Type
2ap40-1	40	[0,19]	Full random
2ap50-1	50	[0,19]	(Tuytens, see [30])
2APt40-r100	40	[0,99]	Negatively correlated
2APt50-r100	50	[0,99]	(Pedersen, see [28])
2AP40-1C100	40	[1,100]	With repetitions
2AP50-1C100	50	[1,100]	(Degoutin, see [7])

Table 7

Six selected numerical instances: results (N.A.=not available).

Name	Y_N	Y_{NS}	Y_{NN}	CPUt/exact	CPUt/heuristic (s)	M1 (%)
2ap40-1	127	38	89	0.15 s	5.64	92
2ap50-1	163	45	118	0.31 s	8.98	87
2APt40-r100	1474	51	1423	159.38 s	268.31	88
2APt50-r100	1969	65	1904	4537.02 s	496.67	73
2AP40-1C100	4	3	1	N.A.	24.45	100
2AP50-1C100	2	2	0	N.A.	85.23	100

generate the solutions for the Degoutin instances.³ In analyzing the characteristics of these instances, the difficulties are easily explained:

- *Tuytens instances*: The common case. For this class of instances, the efficient frontier presents a shape globally convex. The non-dominated points are well distributed along the efficient frontier. The number of non-dominated points is reasonable. With these characteristics, the exact method is very efficient.
- *Pedersen instances*: A first pathologic case. The efficient frontier is in two parts, each part is presenting a flat shape. This characteristic explains the difficulty of the exact method (mainly the second phase is not able to perform efficiently). The number of non-dominated points is large and grows quickly. Y_N is mainly composed by non-supported non-dominated points.
- *Degoutin instances*: A second pathologic case. Y_N contains very few non-dominated points, and very few non-supported non-dominated points. However, for each non-dominated point, it corresponds a very huge number of equivalent solutions, explaining the difficulty of the exact method (all these equivalent solutions has to be enumerated).

The approximation method has no difficulty to process these instances as reported in Fig. 12 and Table 7. The quality indicator M1 measured remains high for the data sets. The CPU time is a step higher for Pedersen instances due to the significant number of non-dominated points to deal with. The few number of points in Y_{NS} implies that the local nadir points are certainly not tight enough, and then many local search are probably triggered with a low positive impact. A possible way to improve the situation consists in replacing local nadir points by heuristic local bounds as proposed in [27].

4. Conclusion

We have described a population-based algorithm for solving assignment problems with two objectives. This multiobjective

³ The exact solutions have been obtained with a ϵ -constraint algorithm using IBM Cplex as solver. Even this algorithm approach is known less efficient than the two phase method for the Tuytens instances, the conclusion is not valid for this class of instances proposed by Degoutin. It is due to the huge number of equivalent solutions to handle.

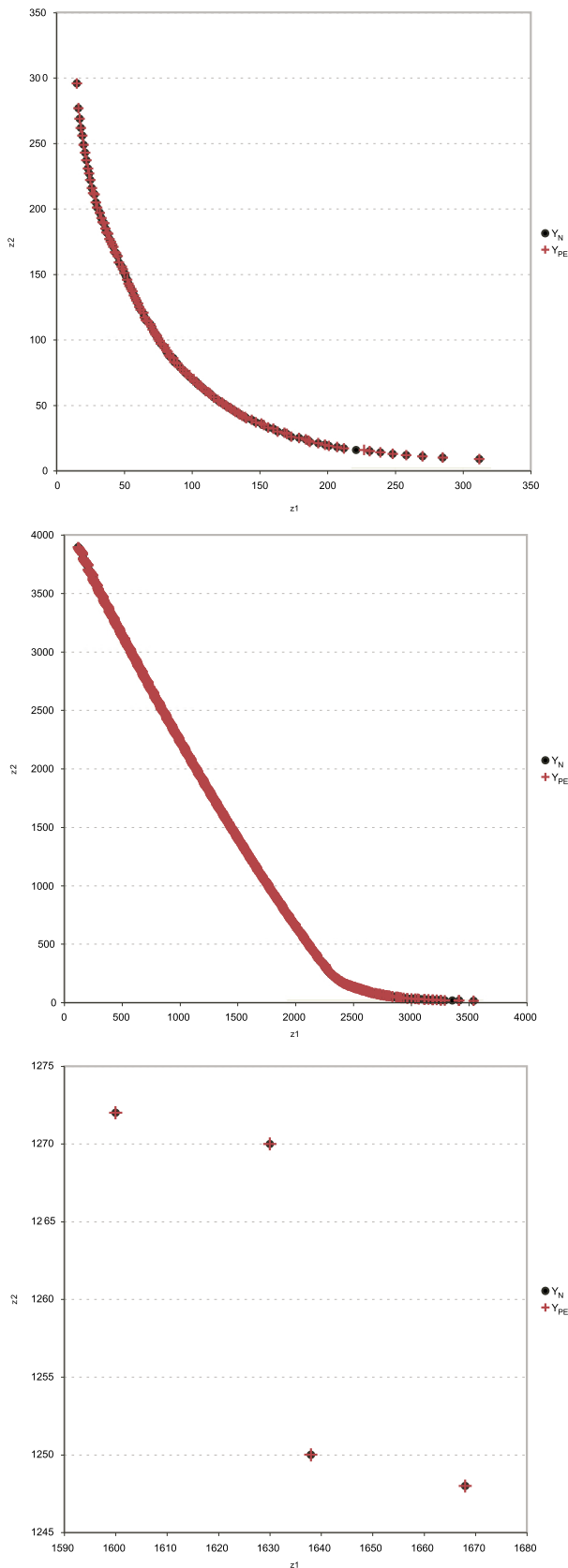


Fig. 12. Non-dominated points Y_N (●) and an approximation Y_{PE} (+). From top to bottom, the plots correspond respectively to the instance 2ap40-1 [30], 2APt40-r100 [28], and 2AP40-1C100 [7].

metaheuristic uses three operators – crossover, path-relinking and local search – on a population composed only of elite solutions. Based on simple principles, our algorithm is easy to implement and involves only two parameters that do not require any tuning phase.

In the first step, a set of efficient supported solutions is computed. This set composes the initial population of elite solutions. In a second step, this population undergoes a generation process involving the three operators. The heuristic is able to detect a good approximation of the efficient frontier even given a short computing time (i.e. a small number of iterations). This is confirmed by experiments that show good performances on numerical instances, especially as compared to the results previously obtained with other heuristics and for pathologic instances.

Several perspectives for further research are possible. First, in order to handle larger instances more efficiently, it may be useful to divide the genetic information into sectors along the efficient frontier, according to a principle of clusters. Such a technique could help to maintain a representative genetic map locally and avoid genetic map sterilized by too many diverse solutions. Another promising track consists in designing of a less random path-relinking operator. Rather than building only one neighbor at random for each stage of the path, the generation of multiple neighbors according to a given characteristic, and the selection of a good neighbor from among them, could make path-relinking even more powerful. The third possibility concerns the local search. In the current version, a local search is systematically applied to a solution when it is included in the promising zone. The same solution can be visited several times and thus generating several times the same neighbors. An improved filtering of the solutions that must undergo a local search would contribute to the reduction of the computation time. Lastly, the resolution ability of our heuristic could be confirmed by further experimentations with pathologic numerical instances exhibiting contrasted characteristics.

Acknowledgments

The authors warmly thank Dr. Anthony Przybylski (University of Nantes, France) for the fruitful discussions held during the revision stage of this paper and for having provided to us the software corresponding to the exact method published in [29].

References

- [1] Ahuja RK, Magnanti TL, Orlin JB. *Network flows: theory, algorithms, and applications*. Upper Saddle River, NJ, USA: Prentice-Hall, Inc.; 1993.
- [2] Baluja S. Population-based incremental learning: a method for integrating genetic search based function optimization and competitive learning (No. CMU-CS-94-163). Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA; 1994.
- [3] Burkard R, Dell'Amico M, Martello S. *Assignment problems*. SIAM; 2009. <http://epubs.siam.org/doi/pdf/10.1137/1.9781611972238>.
- [4] Charnes A, Cooper WW, Niehaus RJ, Stredry A. Static and dynamic assignment models with multiple objectives and some remarks on organization design. *Manag Sci* 1969;15:365–75.
- [5] Coello C, Van Veldhuizen D, Lamont G. *Evolutionary algorithms for solving multi-objective problems*. New York: Kluwer Academic Publishers; 2002.
- [6] Dathe HM. Zur Lösung des Zuordnungsproblems bei zwei Zielgrößen. *Z Oper Res* 1978;22:105–18.
- [7] Degoutin F, Gandibleux X. Un retour d'expériences sur la résolution de problèmes combinatoires bi-objectifs, 5e journée du groupe de travail Programmation Mathématique MultiObjectif (PM20), Angers, France, mai; 2002.
- [8] Delort C. Algorithmes d'énumération implicite pour l'optimisation multi-objectifs: exploitation d'ensembles bornant et application aux problèmes de sac à dos et d'affectation. [Ph.D thesis]. Université Pierre et Marie Curie (Paris VI); 2011.
- [9] Dorigo M, Di Caro G. The ant colony optimization meta-heuristic. In: Corne D,

- Dorigo J, Glover F, editors. In new ideas in optimization. Maidenhead, UK: McGraw-Hill Ltd.; 1999. p. 11–32.
- [10] Ehrgott M. Multicriteria optimization. 2 ed. Berlin, Heidelberg: Springer; 2005.
- [11] Ehrgott M, Gandibleux X. Bounds and bound sets for biobjective combinatorial optimization problems. In: Koksalan M, St. Ziont, editors, Multiple criteria decision making in the new millennium. Lecture notes in economics and mathematical systems, vol. 507. Berlin, Heidelberg: Springer; 2001. p. 241–53.
- [12] Ehrgott M, Gandibleux X. A survey and annotated bibliography of multi-objective combinatorial optimization. *OR Spektr* 2000;22:425–60.
- [13] Ehrgott M, Gandibleux X. Multiobjective combinatorial optimization. In: Ehrgott M, Gandibleux X, editors. Multiple criteria optimization: state of the art annotated bibliographic survey. Kluwer's international series in operations research and management science, vol. 52. Boston: Kluwer Academic Publishers; 2002. p. 369–444.
- [14] Eichfelder G, Gandibleux X, Geiger MJ, Jahn J, Jaszkiwicz A, Knowles JD, et al. Heterogeneous functions. In: Greco S, Klamroth K, Knowles JD, Rudolph G, editors, Dagstuhl reports. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, vol. 5, no. 1. Dagstuhl, Germany; 2015. ISSN: 2192–5283. p. 121–29. (<http://drops.dagstuhl.de/opus/volltexte/2015/5037>).
- [15] Gandibleux X, Morita H, Katoh N. The supported solutions used as a genetic information in a population heuristic. In: Zitzler E, Deb K, Thiele L, Coello C, Corne D, editors, Evolutionary multi-criterion optimization. Lecture notes in computer sciences, vol. 1993. Berlin, Heidelberg: Springer; 2001. p. 429–42.
- [16] Gandibleux X, Morita H, Katoh N. Use of a genetic heritage for solving the assignment problem with two objectives. In: Fonseca C, Fleming P, Zitzler E, Deb K, Thiele L, editors, Evolutionary multi-criterion optimization. Lecture notes in computer sciences, vol. 2632. Berlin, Heidelberg: Springer; 2003. p. 43–57.
- [17] Gandibleux X, Morita H, Katoh N. Impact of clusters, path-relinking and mutation operators on the heuristic using a genetic heritage for solving assignment problems with two objectives. In: MIC2003 Fifth Metaheuristics International Conference, Kyoto, Japan; August 2003. p. 25–8.
- [18] Gandibleux X, Morita H, Katoh N. A population-based metaheuristic for solving assignment problems with two objectives. Technical report n°7/2003/ROI, LAMIH, Université de Valenciennes, France; 2003.
- [19] Gandibleux X, Sevaux M, Sörensen K, T'Kindt V, editors. Metaheuristics for multiobjective optimisation. In: Proceedings of the workshop "MOMH: Multiple Objective MetaHeuristics"; November 4–5, 2002, Carré des Sciences, Paris. Lecture notes in economics and mathematical systems, vol. 535. Berlin, Heidelberg: Springer; 2004. p. 249.
- [20] Glover F, Laguna M. Tabu search. Boston: Kluwer Academic Publishers; 1997.
- [21] Hansen P. Bicriterion path problems. In: Fandel G, Gal T, editors, Multiple criteria decision making theory and application. Lecture notes in economics and mathematical systems, vol. 177. Berlin: Springer-Verlag; 1979. p. 109–127.
- [22] Kuhn HW. The Hungarian method for the assignment problem. *Naval Res Logist Q* 1955;2:83–97.
- [23] Malhotra R, Bhatia HL, Puri MC. Bi-criteria assignment problem. *Oper Res* 1982;19(2):84–96.
- [24] Morita H, Gandibleux X, Katoh N. Experimental feedback on biobjective permutation scheduling problems solved with a population heuristic. *Found Comput Dec Sci J* 2001;26(1):23–50.
- [25] Muehlenbein H, Paass G. From recombination of genes to the estimation of distributions. In: Voigt HM, et al., editors, Parallel problem solving from nature —PPSN IV. Lecture notes in computer science, vol. 1141; 1996. p. 178–87.
- [26] Muehlenbein H. The equation for response to selection and its use for prediction. *Evol Comput* 1998;5:303–46.
- [27] Pasia J, Gandibleux X, Doerner K, Hartl R. Local search guided by path relinking and heuristic bounds. In: Obayashi Sh, Deb K, Poloni C, Hiroyasu T, Murata T, editors, Evolutionary multi-criterion optimization. Lecture notes in computer science, vol. 4403. Berlin, Heidelberg: Springer; 2007. p. 501–15.
- [28] Pedersen ChR, Nielsen LR, Andersen KA. The bicriterion multimodal assignment problem: introduction, analysis, and experimental results. *INFORMS J Comput* 2008;20:400–11.
- [29] Przybylski A, Gandibleux X, Ehrgott M. Two phase algorithms for the bi-objective assignment problem. *Eur J Oper Res* 2008;185(2):509–33.
- [30] Tuytens D, Teghem J, Fortemps Ph, Van Nieuwenhuysse K. Performance of the MOSA method for the bicriteria assignment problem. *J Heuristics* 2000;6:295–310.
- [31] Ulungu EL, Teghem J, Fortemps P, Tuytens D. MOSA method: a tool for solving multi-objective combinatorial optimization problems. *J Multi-Criteria Dec Anal* 1999;8(4):221–36.
- [32] Ulungu EL, Teghem J. The two-phases method: an efficient procedure to solve bi-objective combinatorial optimization problems. *Found Comput Dec Sci* 1994;20(2):149–65.